

C++继承与多态 - 继承多态原理01

原创 VIP免费 已于 2024-03-24 21:53:06 修改 · 2.9k 阅读 ·  20 ·  14

知识的学习在于点滴记录，坚持不懈；知识的学习要有深度和广度，不能只流于表面，坐井观天；知识要善于总结，不仅能够理解，更知道如何考

目录

- 继承简介
- 派生类对象的构造过程
- 重载、隐藏、覆盖
- 虚函数、vfp_{tr}和vftable
- 静态绑定和动态绑定
- 多态
- 抽象类
- 虚析构函数

继承简介

继承的好处是什么？

- 1.基类 给所有派生类提供公共的属性（成员变量）和方法（成员函数），通过继承达到代码复用的目的。
- 2.基类可以给所有派生类提供统一的纯虚函数接口，派生类通过函数重写，达到多态调用的目的。（OOP的很多设计模式离不开继承和多态，为了达件设计，如高内聚，低耦合，遵循‘开-闭’原则等，继承和多态是必须涉及到的）。

OOP面向对象语言如C++，Java，Python，PHP等，非常重要的一项语言特征就是继承，继承首先可以做到代码的复用，派生类通过继承基类，可以的成员变量和成员方法，那么这里的第一个问题就是派生类从基类继承了那么多成员，其访问限定是怎么样子的呢？如下代码示例：

java 用码道免费领 1 个月 Token

```
// A是基类
class A
{
public:
    int ma;
protected:
    int mb;
private:
    int mc;
};
// B是派生类，继承到派生类里面的成员访问限定要小于等于继承方式
class B : public A
{
public:
    int md;
protected:
    int me;
private:
    int mf;
};
```

派生类B从基类A继承了成员ma，mb和mc，那么这三个成员在派生类B种的访问权限是什么，看如下表格：

继承方式	基类成员访问限定	在派生类种的访问限定	外部函数中的访问限定
public	public	public	在外部可以访问
public	protected	protected	在外部不可以访问
public	private	派生类中不可见	在外部不可以访问

继承方式	基类成员访问限定	在派生类种的访问限定	外部函数中的访问限定
protected	public	protected	在外部不可以访问
protected	protected	protected	在外部不可以访问
protected	private	派生类中不可见	在外部不可以访问

继承方式	基类成员访问限定	在派生类种的访问限定	外部函数中的访问限定
private	public	private	在外部不可以访问
private	protected	private	在外部不可以访问
private	private	派生类中不可见	在外部不可以访问

从上面的总结可以看出：

1.基类的private 私有成员，无论采用什么继承方式，在派生类里面都是可以继承下来，但是无法访问。

2.protected 和private的区别。基类的private私有成员，在派生类和外部都不能直接访问；但是基类protected的成员，在派生类中是可以访问的，在问。

派生类对象的构造过程

派生类从基类继承的成员变量该如何初始化 呢？ 必须通过调用基类的构造函数来初始化从基类继承来的成员！，如下代码示例：

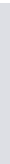
cpp

用码道免费领 1 个月 Token

```
#include <iostream>
using namespace std;

class Base
{
public:
    Base(int data) :ma(data) { cout << "Base()" << endl; }
    ~Base() { cout << "~Base()" << endl; }
protected:
    int ma;
};
class Derive : public Base
{
public:
    // 必须通过调用基类的构造函数来初始化从基类继承来的成员
    Derive(int data) :Base(data) { cout << "Derive()" << endl; }
    ~Derive() { cout << "~Derive()" << endl; }
private:
    int mb;
};
int main()
{
    Derive d(10);
    return 0;
}
```

代码运行打印如下：



Base()
Derive()
~Derive()
~Base()

可以看到，一个派生类对象的构造和析构顺序是：

1.先调用基类构造函数 ，构造从基类继承来的成员

2.再调用派生类自己的构造函数，构造派生类自己的成员

3.先调用派生类自己的析构函数，释放派生类自己占用的外部资源

4.调用基类的析构函数，释放基类部分成员占用的外部资源

重载、隐藏、覆盖

基类和派生类中，是否可以定义同名的成员（包括成员变量和成员方法名字）呢？答案是可以的。可以这样理解，派生类从基类继承来的成员，都带作用域，作用域不同，名字相同的成员是不会冲突的。

基类和派生类之间的**同名成员方法**有三种关系：**重载**，**隐藏**和**覆盖**。如下代码示例：

cpp

用码道免费领 1 个月 Token

```
#include <iostream>
using namespace std;

class Base
{
public:
    Base(int data) :ma(data) { cout << "Base()" << endl; }
    ~Base() { cout << "~Base()" << endl; }
    void show() { cout << "Base::show()" << endl; } // @1
    void show(int) { cout << "Base::show(int)" << endl; } // @2
protected:
    int ma;
};

class Derive : public Base
{
public:
    Derive(int data) :Base(data) { cout << "Derive()" << endl; }
    ~Derive() { cout << "~Derive()" << endl; }
    void show() { cout << "Derive::show()" << endl; } // @3
private:
    int mb;
};

int main()
{
    Derive d(10);
    d.show(); // 调用的是派生类自己的show方法
    d.Base::show(); // 指定了作用域，调用的是派生类从基类继承来的show方法
    //d.show(10); // 编译错误，不能调用，派生类的show方法把基类继承来的show方法给隐藏了
    return 0;
}
```

通过上面的代码示例，给出重载，隐藏和覆盖的概念：

重载：一组函数**必须在同一个作用域中，函数名相同，参数列表不同，才能称做函数重载**，所以上面的代码示例中@1和@2是重载函数；@2和@3虽同，参数列表不同，但它们不在同一个作用域，不是重载关系。

隐藏：指的是基类和派生类中的同名成员，只要成员名字相同，用派生类对象调用该成员名字时，就发生了隐藏（**隐藏了基类的同名成员**），派生类都是自己的成员，如果要调用基类的同名成员，需要添加基类的作用域。上面代码中@1和@3、@2和@3都是隐藏关系。

覆盖：指的是基类和派生类中的同名成员函数，**不仅函数名字相同、返回值相同、参数列表也相同，而且基类的该函数是virtual虚函数**，那么派生类自动被处理成虚函数，它们之间的关系是覆盖关系。上面代码中没有覆盖关系的函数，看下面的代码@1和@3就是覆盖关系（**覆盖主要指的是虚函数址的覆盖重写**）。

虚函数、vfptr和vftable

看下面代码示例：

cpp

用码道免费领 1 个月 Token

```
class Base
{
public:
    Base(int data) :ma(data) { cout << "Base()" << endl; }
    ~Base() { cout << "~Base()" << endl; }
    // 下面这个show()函数是虚函数
    virtual void show() { cout << "Base::show()" << endl; } // @1
    // 下面这个show(int)函数是虚函数
    virtual void show(int) { cout << "Base::show(int)" << endl; } // @2
    // 下面这个show(int, int)函数是普通函数
    void show(int, int) { cout << "Base::show(int, int)" << endl; }
protected:
    int ma;
};

class Derive : public Base
```

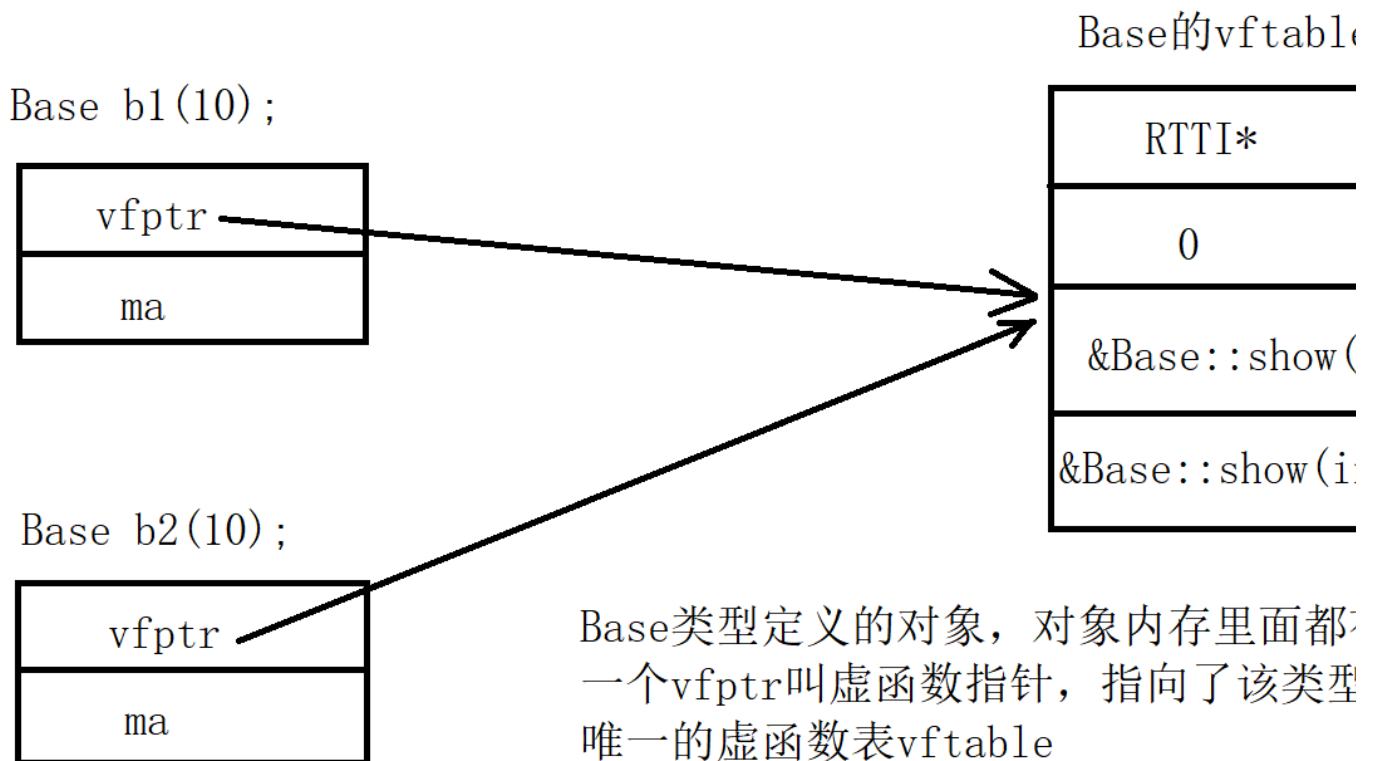
```

{
public:
    Derive(int data) :Base(data) { cout << "Derive()" << endl; }
    ~Derive() { cout << "~Derive()" << endl; }
    void show() { cout << "Derive::show()" << endl; } // @3
private:
    int mb;
};

```

上面代码中，基类Base 里面有虚函数，这个编译器 是怎么处理的？处理过程如下：如果一个类有虚函数，那么编译器在编译阶段，会给当前类生成虚函数表，虚函数表里面放的是虚函数的地址，还有RTTI指针信息；这张虚函数表在代码运行阶段，被加载到.rodata段，也就是说只读不能写，这张表就是vftable。

那么用这个Base类型实例化的对象，是怎么找到该类型相应的vftable的呢？在对象的前4个字节里面，存储一个vfptr，叫虚函数指针，里面存放的就是vftable虚函数表的地址。如下图所示：



<https://blog.csdn.net/QIANGWEIYUAN>

用VS自带的命令行工具，也可以查看对象的内存布局，如下：

D:\代码\C++ Code\博客代码\继承>cl 继承.cpp /d1reportSingleClassLayoutBase

显示如下：

```

class Base      size(8):
    +---+
    0      {vfptr} 虚函数指针
    4      ma
    +---+

Base::$vftable@:
    &Base_meta RTTI指针
    0
    0      &Base::show
    1      &Base::show 虚函数的地址

```

<https://blog.csdn.net/QIANGWEIYUAN>

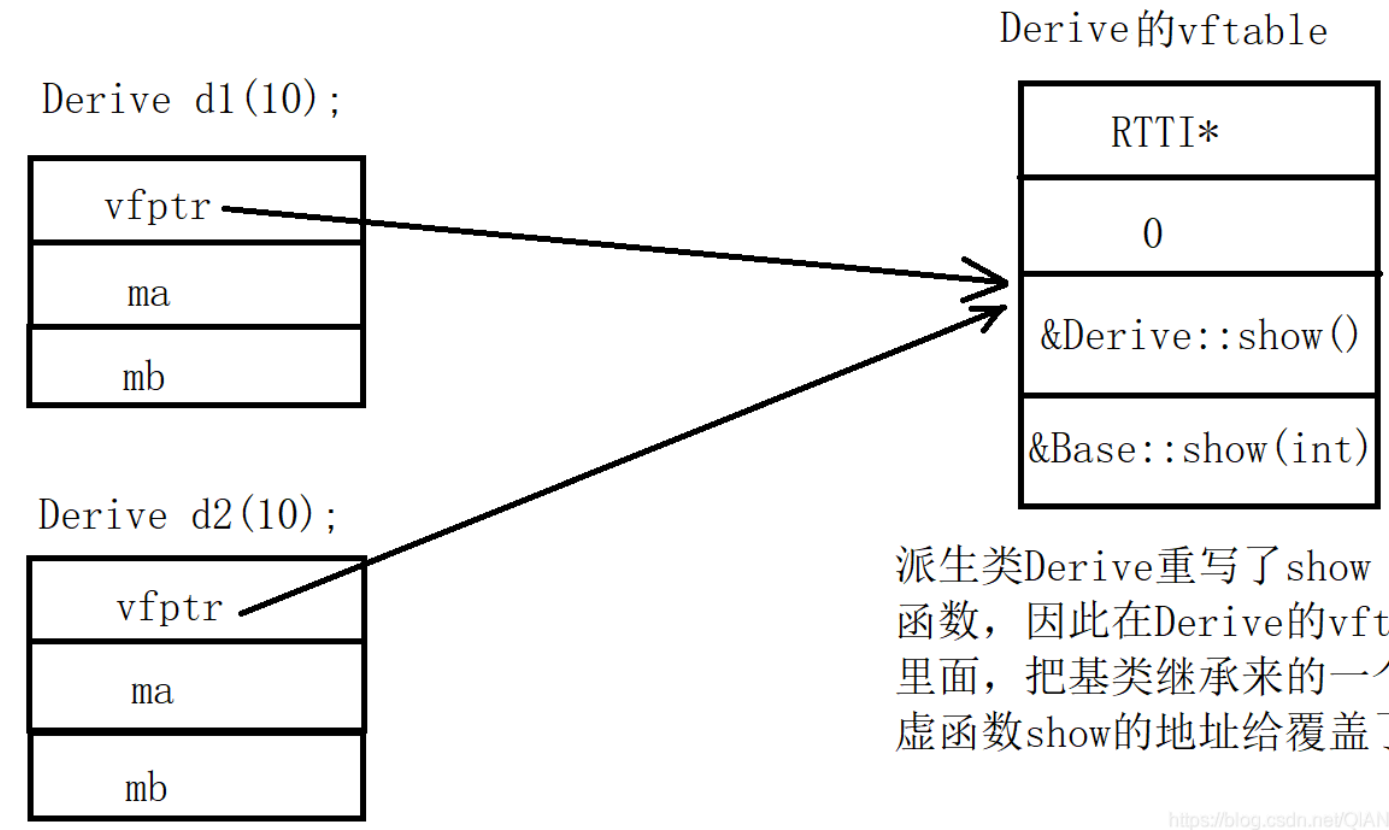
可以看到，命令显示的内容和上面图片上的内容是一致的。说完了基类Base，现在看一下Derive派生类的处理过程。

Derive派生类在编译过程中，产生的虚函数表本来放的是从基类Base继承的两个虚函数地址，但是Derive本身提供了一个覆盖函数，如下：

cpp

void show() { cout << "Derive::show()" << endl; } // @3

所以，Derive派生类对象的内存布局如下：



<https://blog.csdn.net/QIAN>

上面演示了在继承结构中，如果类里面出现了虚函数，都产生了哪些影响，总结如下：

- 1.类中出现虚函数，编译阶段会给该类型产生虚函数表，里面存放了虚函数的地址和RTTI指针。
- 2.有虚函数的类实例化的对象，内存都多了一个vfptr虚函数指针，指向该对象类型的虚函数表，同类型对象都有自己的vfptr，但是它们共享一个vftable。
- 3.派生类如果提供了同名覆盖函数，那么在虚函数表中，需要把基类继承来的虚函数地址给覆盖掉。
- 4.一个类里面出现多个虚函数，对象的内存只增长4个字节（vfptr），但是虚函数表的大小会逐渐增大。

静态绑定和动态绑定

这里的绑定，指的是函数调用。静态绑定指编译时期函数的调用就是确定的；动态绑定指函数的调用要到运行时期才能确定，动态绑定是实现OOI调用的技术基础，否则无法实现多态。

看如下代码示例，理解静态绑定和动态绑定：

cpp

#include <iostream>
using namespace std;

class Base
{
public:
 Base(int data) :ma(data) { cout << "Base()" << endl; }
 ~Base() { cout << "~Base()" << endl; }
 // 下面这个show()函数是虚函数，为了演示动态绑定
 virtual void show() { cout << "Base::show()" << endl; }
 // 下面这个show(int, int)函数是普通函数，为了演示静态绑定
 void show(int, int) { cout << "Base::show(int, int)" << endl; }
protected:
 int ma;
};

class Derive : public Base
{
public:
 Derive(int data) :Base(data) { cout << "Derive()" << endl; }

```

~Derive() { cout << "~Derive()" << endl; }
void show() { cout << "Derive::show()" << endl; }
private:
    int mb;
};
int main()
{
    Derive d(10);
    Base *p = &d;
    /*
        这里p的类型是Base, Base下的show(int,int)是普通函数, 这里是静态绑定,
        就是编译阶段就确定了, 调用的是Base::show(int, int)函数, 可以通过其
        汇编指令查看是call Base::show...
    */
    p->show(0, 0);
    /*
        这里p的类型是Base, Base下的show()是虚函数, 这里是动态绑定, 其指令首先
        访问p指向的对象的前4个字节vfptr, 再访问对应的vftable, 然后从vftable
        中取虚函数的地址进行函数调用, 只有在运行的时候, 才能取到虚函数表中存放的
        函数地址, 因此是运行时的绑定, 称作动态函数绑定
    */
    p->show();

    cout << sizeof(Base) << endl; // 8
    cout << sizeof(Derive) << endl; // 12
    /*
        这里p的类型是Base, 因为Base里面有虚函数, 这里*p识别的就是RTTI类型 (识别RTTI
        类型也是通过指针访问对象内存的vfptr, 再访问vftable, 通过RTTI指针取RTTI类型信息);
        如果p的类型Base里面没有虚函数, 那么*p识别的就是编译时期的类型。
    */
    cout << typeid(*p).name() << endl; // class Derive

    return 0;
}

```

多态

多态指的是, 用基类指针 (同引用) 指向从它继承的一组派生类对象, 调用派生类的同名覆盖方法, **基类指针指向哪个派生类对象, 就会调用相应派生类同名覆盖方法**, 怎么做到的呢? 因为通过基类指针调用派生类的同名覆盖方法时, **发生了动态绑定**, 访问了基类指针指向对象的虚函数表vftable, 从中取出来的就是这个派生类重写的虚函数的地址, 然后进行调用, 当然就做到了基类指针指向谁, 就能调用谁的方法了 (因为指向谁, 就访问谁的虚函数了)。

可以参考我的另一篇介绍工厂方法和抽象工厂的设计模式, 就是多态的一个经典的应用, 博客地址: <https://blog.csdn.net/QIANGWEIYUAN/article/details/88792594>

抽象类

从定义上说, 拥有纯虚函数的类就是抽象类, 如下所示:

cpp

用码道免费领 1 个月 Token

```

class Animal
{
public:
    Animal(string name) :_name(name) {}
    virtual void bark() = 0; // 纯虚函数
protected:
    string _name;
};

```

上面代码的Animal类就是一个抽象类, 抽象类不能定义对象, 但是可以定义指针或者引用。一般把基类往往设计成抽象类, 因为定义基类的初衷, 并它去抽象化某个实体的类型, 而是刚开始我们讲过的原因, 如下:

- 1.基类给所有派生类提供公共的属性 (成员变量) 和方法 (成员函数), 通过继承达到代码复用的目的。
- 2.基类可以给所有派生类提供统一的纯虚函数接口, 派生类通过函数重写, 达到多态调用的目的。

就像上面的Animal类, 它并不代表某一个具体的动物类型, 而是所有动物类型的泛指, 因此对于这样的类Animal, 其bark方法根本无法提供具体实现, bark设置成纯虚函数, 等待从Animal继承的派生类中, 对bark方法进行重写 (因为派生类型就是对具体的某个动物的类型说明了)。

虚析构函数

析构函数肯定是可以实现成虚函数的，因为析构函数调用的时候对象是存在的。那什么时候析构函数要定义成虚析构函数呢？先看如下的代码：

cpp

用码道免费领 1 个月 Token

```
class Base // 基类定义
{
public:
    Base(int data=10):_ptrb(new int(data))
    { cout << "Base()" << endl; }
    ~Base() { delete _ptrb; cout << "~Base()" << endl; }
protected:
    int *_ptrb;
};
class Derive : public Base // 派生类定义
{
public:
    Derive(int data=20):Base(data), _ptrd(new int(20))
    { cout << "Derive()" << endl; }
    ~Derive() { delete _ptrd; cout << "Derive()" << endl; }
private:
    int *_ptrd;
};
int main()
{
    Base *p = new Derive();
    delete p; // 只调用了Base的析构函数，没有调用Derive派生类的析构函数
    return 0;
}
```

main函数运行代码打印如下：

```
Base()
Derive()
~Base()
```

很明显的错误就是delete p的时候，**派生类的析构函数没有调用**，因为p是基类Base类型，而且基类中的析构函数~Base是一个普通函数，**所以delete进行了静态绑定，直接调用了基类的析构函数，派生类的析构函数根本没有机会调用**，如果派生类的析构函数中有释放外部资源的代码，那么上面的在内存泄漏了！

如何修改？**需要把基类的析构函数实现成虚析构函数**，这时候delete p编译的时候，看到p的类型是Base，Base里面的析构函数是虚析构函数，那么d
构函数的调用就是动态绑定了，因为p指针指向的是一个派生类对象，**那么最终从派生类Derive的虚函数表中，取得的就是派生类的重写的析构函数的**
此派生类的析构函数就得以调用，代码修改如下：

cpp

用码道免费领 1 个月 Token

```
class Base // 基类定义
{
public:
    Base(int data=10):_ptrb(new int(data))
    { cout << "Base()" << endl; }
    // 定义虚析构函数
    virtual ~Base() { delete _ptrb; cout << "~Base()" << endl; }
protected:
    int *_ptrb;
};
class Derive : public Base // 派生类定义
{
public:
    Derive(int data=20):Base(data), _ptrd(new int(20))
    { cout << "Derive()" << endl; }
    ~Derive() { delete _ptrd; cout << "Derive()" << endl; }
private:
    int *_ptrd;
};
int main()
{
    Base *p = new Derive();
    delete p; // 此处析构函数调用是动态绑定了，调用正确！
}
```

```
    return 0;  
}
```

代码运行如下：

```
Base()  
Derive()  
~Derive()  
~Base()
```

可以看到，析构函数调用完全正确，派生类和基类的析构函数都调用了。所以结论就是，一般基类的析构函数要实现成虚析构函数，当基类指针指向类对象，释放资源的时候使用`delete p`操作，此时析构函数的调用就是一个动态的函数绑定调用，派生类的析构函数和基类的析构函数就都可以调用。



CSDN @大秦坑王

标签 [#继承](#) [#多态](#)